

```
/**
 * server/services/dead-letter-drain.ts (final – all 7 delta items applied)
 *
 * Fix #6: per-item channel/token health check (not global block for any reconnect)
 * Fix #7: transactional insert+update; next_retry_at set on budget-blocked items
 * All previous fixes retained: rate limit (1/12min), singleton, CommandCenter,
 * YouTube-only, phantom permanent_fail, budget map, jsonb cast, expired_at tracking
 */
```

```
import { db } from '../db.js';
import { sql } from 'drizzle-orm';
import { createLogger } from '../lib/logger.js';
import { setJitteredInterval } from '../lib/timer-utils.js';
import { CommandCenter } from '../lib/command-center.js';
import { isProductionAutomationAllowed } from '../lib/production-guard.js';
```

```
const log = createLogger('dead-letter-drain');
```

```
// — Singleton lock —————
```

```
let isRunning = false;
```

```
// — Content-type → required module budget map —————
```

```
const BUDGET_MAP: Record<string, string[]> = {
  'youtube_short': ['shorts-pipeline', 'viral-optimizer'],
  'platform_short': ['shorts-pipeline', 'viral-optimizer'],
  'vod-short': ['repurpose-engine', 'shorts-pipeline'],
  'auto-clip': ['repurpose-engine', 'shorts-pipeline'],
  'smart-edit': ['vod-seo-optimizer'],
  'thumbnail': ['auto-thumbnail'],
  'long-form-clip': ['viral-optimizer'],
  'long-form-compilation': ['viral-optimizer'],
};
```

```
async function getExhaustedBudget(contentType: string): Promise<string | null> {
  const modules = BUDGET_MAP[contentType] ?? ['viral-optimizer'];
  for (const mod of modules) {
    try {
      const r = await db.execute<{ used: string; daily_limit: string }>(sql`
        SELECT used_tokens AS used, daily_limit
        FROM token_budgets WHERE module = ${mod}
      `);
      const row = r.rows[0] as any;
      if (row && parseInt(row.used) >= parseInt(row.daily_limit)) return mod;
    } catch { /* unavailable – treat as ok */ }
  }
  return null;
}
```

```

}

// — Per-item channel health check (Fix #6) —————
// Do NOT use a global "any channel needs reconnect" block.
// Check the specific user's YouTube channel for each DLQ item.

async function itemChannelIsHealthy(userId: string): Promise<{
  ok:      boolean;
  reason:  string;
}> {
  try {
    const result = await db.execute<{
      access_token:      string | null;
      refresh_token:     string | null;
      needs_reconnect:   boolean;
      token_expires_at:  Date | null;
    }>(sql`
      SELECT access_token, refresh_token, needs_reconnect, token_expires_at
      FROM channels
      WHERE user_id = ${userId}
        AND platform = 'youtube'
      LIMIT 1
    `);

    const ch = result.rows[0];
    if (!ch)          return { ok: false, reason: 'No YouTube channel for this user' };
    if (ch.needs_reconnect) return { ok: false, reason: 'Channel needs reconnect' };
    if (!ch.refresh_token) return { ok: false, reason: 'refresh_token is null' };
    if (!ch.access_token)  return { ok: false, reason: 'access_token is null' };

    const expiresAt = ch.token_expires_at ? new Date(ch.token_expires_at) : null;
    if (expiresAt && expiresAt <= new Date()) {
      // Expired access token but refresh token present – caller can refresh;
      // allow requeue (refresh happens at publish time via channelCanPublish)
      return { ok: true, reason: 'access token expired but refresh token present – ok to
requeue' };
    }

    return { ok: true, reason: 'healthy' };
  } catch {
    return { ok: false, reason: 'Channel health check failed' };
  }
}

// — Global quota guard only —————
// (channel-specific health is per-item above; only quota is truly global)

async function quotaBreakerActive(): Promise<boolean> {
  try {
    const q = await db.execute<{ tripped: boolean }>(sql`

```

```

SELECT tripped FROM youtube_quota_state ORDER BY recorded_at DESC LIMIT 1
`);
return q.rows[0]?.tripped ?? false;
} catch { return false; }
}

// — One-time triage —————

let triageDone = false;

async function triageOnce(): Promise<void> {
  if (triageDone) return;
  triageDone = true;
  const result = await db.execute(sql`
    UPDATE dead_letter_queue
    SET status          = 'expired',
        expired_at      = NOW(),
        error_message    = 'Expired: content older than 30 days'
    WHERE status NOT IN ('expired', 'requeued', 'resolved')
        AND created_at < NOW() - INTERVAL '30 days'
  `);
  const count = result.rowCount ?? 0;
  if (count > 0) log.info(`[DeadLetterDrain] Expired ${count} stale item(s)`);
}

// — Single-item drain (Fix #7: transactional; next_retry_at on budget block) —

async function drainOneCycle(): Promise<void> {
  if (isRunning) {
    log.debug('[DeadLetterDrain] Still running – skipping');
    return;
  }
  isRunning = true;

  try {
    await triageOnce();

    if (await quotaBreakerActive()) {
      log.debug('[DeadLetterDrain] Quota breaker active – skipping cycle');
      return;
    }

    const approved = await CommandCenter.requestApproval('dead_letter_drain_scan', {
      description: 'Drain one item from dead_letter_queue',
    }).catch(() => false);
    if (!approved) {
      log.debug('[DeadLetterDrain] CommandCenter denied scan');
      return;
    }
  }
}

```

```

// Pick one eligible item
// Fix #7: filter next_retry_at so budget-blocked items don't loop forever
const items = await db.execute<{
  id:                number;
  content_type:       string;
  user_id:            string;
  platform:           string | null;
  original_queue_item_id: number | null;
  payload:            unknown;
  created_at:         Date;
  requeue_count:      number;
}>(sql`
SELECT
  dlq.id,
  dlq.content_type,
  dlq.user_id,
  dlq.platform,
  dlq.original_queue_item_id,
  dlq.payload,
  dlq.created_at,
  COALESCE(dlq.requeue_count, 0) AS requeue_count
FROM dead_letter_queue dlq
WHERE dlq.status NOT IN ('expired', 'requeued', 'resolved')
  AND COALESCE(dlq.platform, 'youtube') = 'youtube'
  AND (dlq.next_retry_at IS NULL OR dlq.next_retry_at <= NOW())
  AND dlq.user_id NOT IN (
    'tiktok_-000hfXLzkfKJGE24wvR-qZP9Pw6iwxLWyeM',
    '54374239',
    'google_api_demo_reviewer',
    'phase1-done-criteria-user'
  )
  AND LOWER(COALESCE(dlq.user_id, '')) NOT LIKE '%demo%'
  AND LOWER(COALESCE(dlq.user_id, '')) NOT LIKE '%test%'
  AND LOWER(COALESCE(dlq.user_id, '')) NOT LIKE '%reviewer%'
ORDER BY
  CASE dlq.content_type
    WHEN 'youtube_short' THEN 1
    WHEN 'platform_short' THEN 1
    WHEN 'vod-short'     THEN 2
    WHEN 'auto-clip'     THEN 2
    ELSE 3
  END ASC,
  dlq.created_at ASC
LIMIT 1
`);

if (items.rows.length === 0) {
  log.debug('[DeadLetterDrain] No eligible items');
  return;
}

```

```

const item = items.rows[0];

// Fix #8: centralized production guard replaces local phantom set
const prodGuard = isProductionAutomationAllowed(item.user_id, undefined, item.platform ??
undefined);
if (!prodGuard.allowed) {
  await db.execute(sql`
    UPDATE dead_letter_queue
    SET status          = 'resolved',
        error_message = '${'Blocked by production guard: ' + prodGuard.reason},
        updated_at     = NOW()
    WHERE id = ${item.id}
  `);
  log.debug(`[DeadLetterDrain] DLQ item ${item.id} blocked: ${prodGuard.reason}`);
  return;
}

// Fix #6: per-item channel health check
const channelHealth = await itemChannelIsHealthy(item.user_id);
if (!channelHealth.ok) {
  log.debug(`[DeadLetterDrain] Item ${item.id} channel not healthy:
${channelHealth.reason}`);
  // Set next_retry_at so we don't check this item every 12 min
  await db.execute(sql`
    UPDATE dead_letter_queue
    SET next_retry_at = NOW() + INTERVAL '2 hours',
        error_message = ${channelHealth.reason},
        updated_at     = NOW()
    WHERE id = ${item.id}
  `);
  return;
}

// Fix #7: per-content-type budget check; set next_retry_at if blocked
const exhaustedModule = await getExhaustedBudget(item.content_type);
if (exhaustedModule) {
  log.debug(`[DeadLetterDrain] Item ${item.id} budget blocked: ${exhaustedModule}`);
  // Set next_retry_at to midnight (budget reset time) so item is skipped until then
  await db.execute(sql`
    UPDATE dead_letter_queue
    SET next_retry_at = (NOW()::date + INTERVAL '1 day')::timestampz,
        error_message = ${exhaustedModule + ' budget exhausted - retry after reset'},
        updated_at     = NOW()
    WHERE id = ${item.id}
  `);
  return;
}

// Duplicate check

```

```

if (item.original_queue_item_id) {
  const existing = await db.execute<{ status: string }>(sql`
    SELECT status FROM autopilot_queue
    WHERE id = ${item.original_queue_item_id}
    AND status IN ('published', 'pending', 'scheduled', 'processing')
  `);
  if (existing.rows.length > 0) {
    await db.execute(sql`
      UPDATE dead_letter_queue SET status = 'resolved', updated_at = NOW()
      WHERE id = ${item.id}
    `);
    log.info(`[DeadLetterDrain] DLQ item ${item.id} already active – resolved`);
    return;
  }
}

// CommandCenter requeue approval
const actionApproved = await CommandCenter.requestApproval('dead_letter_requeue', {
  dlqId: item.id, contentType: item.content_type,
}).catch(() => false);
if (!actionApproved) {
  log.debug(`[DeadLetterDrain] CommandCenter denied requeue of item ${item.id}`);
  return;
}

const ageDays = Math.round(
  (Date.now() - new Date(item.created_at).getTime()) / 86_400_000
);

// Build enriched payload with traceability embedded
const enrichedPayload = {
  ...(typeof item.payload === 'object' && item.payload !== null ? item.payload : {}),
  recoveredFromDeadLetter: true,
  deadLetterId: item.id,
  originalQueueItemId: item.original_queue_item_id,
  recoverySource: 'dead_letter_drain',
};

// Fix #6: include platform, source, and traceability columns where they exist.
// If columns are missing the INSERT will error – adjust to match your actual schema.
// Columns: platform, target_platform, source, original_queue_item_id, dead_letter_id
// are included as real columns here. If any don't exist, move them into payload only.
await db.transaction(async (tx) => {
  await tx.execute(sql`
    INSERT INTO autopilot_queue (
      type,
      status,
      user_id,
      platform,
      target_platform,

```

```

        payload,
        scheduled_at,
        created_at,
        updated_at,
        source,
        original_queue_item_id,
        dead_letter_id
    ) VALUES (
        ${item.content_type},
        'pending',
        ${item.user_id},
        'youtube',
        'youtube',
        ${JSON.stringify(enrichedPayload)}::jsonb,
        NOW(),
        NOW(),
        NOW(),
        'dead_letter_drain',
        ${item.original_queue_item_id ?? null},
        ${item.id}
    )
`);

await tx.execute(sql`
    UPDATE dead_letter_queue
    SET status          = 'requeued',
        requeued_at    = NOW(),
        requeue_count  = ${item.requeue_count + 1},
        next_retry_at  = NULL,
        updated_at     = NOW()
    WHERE id = ${item.id}
`);
});

log.info(
    `[DeadLetterDrain] ✅ Requeued DLQ item ${item.id} ` +
    `(${item.content_type}, ${ageDays}d old, requeue #${item.requeue_count + 1})`
);

} finally {
    isRunning = false;
}
}

// — Lifecycle —————

let stopInterval: (() => void) | null = null;

export function startDeadLetterDrain(): void {
    if (stopInterval) return;

```

```
log.info('[DeadLetterDrain] Starting - 1 item per 12-min cycle (max 5/hour)');
stopInterval = setJitteredInterval(
  () => drainOneCycle().catch(err =>
    log.error('[DeadLetterDrain] Cycle error:', err)
  ),
  12 * 60 * 1000,
);
}

export function stopDeadLetterDrain(): void {
  if (stopInterval) { stopInterval(); stopInterval = null; }
}
```